

LUNGS CANCER DETECTION

*Project submitted in partial fulfilment for the requirement of the degree of Bachelor of
Technology in Computer Science and Engineering / Information Technology*

By

GOBINDA PANDA

(Registration No.: 2102040002)

SUGYANI MANJARI DASH

(Registration No.: 2102081031)

Under the Guidance of

Mr. Suresh Kumar Srichandan

(Associate Professor)



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,
VEER SURENDRA SAI UNIVERSITY OF TECHNOLOGY,
BURLA, SAMBALPUR – 7682018
[2024-25]**



CERTIFICATE OF EXAMINATION

This is to certify that the project entitled “**Lungs Cancer Detection**” submitted by **Mr. Gobinda Panda (2102040002)** of **Computer Science and Engineering** & **Ms. Sugyani Manjari Dash (2102081031)** of **Information Technology** Branch, Veer Surendra Sai University of Technology, Odisha has been examined by us. We are satisfied with the quality and correctness of the work.

Internal Examiners

HOD, Department of
CSE, VSSUT, Burla



SUPERVISOR'S CERTIFICATE

This is to certify that the Project entitled “**Lungs Cancer Detection**” is being submitted by **Mr. Gobinda Panda (2102040002)** of Computer Science and Engineering & **Ms. Sugyani Manjari Dash (2102081031)** of Information Technology, to the Department of Computer Science and Engineering, Veer Surendra Sai University of Technology, Burla, in partial fulfilment of the requirement for the degree of Bachelor of Technology in Computer Science and Engineering/Information Technology during the academic year 2024-2025. It is an original report carried out by them under my supervision.

Supervisor

Mr. Suresh Kumar Srichandan

(Assistant Professor)

Contents

Title Page	I
Certificate of Examination	II
Supervisor's Certificate	III
Acknowledgement	IV
Abstract	V
1. INTRODUCTION	1
2. LITERATURE REVIEW	2
3. RELATED WORK	3
4. LIBRARIES USED	4
5. VARIOUS STAGES OF PROJECT	6
5.1. Data Collection	6
5.2. Data Preprocessing	7
5.3. Model Training	8
5.4. Output	12
6. CONCLUSION	19
7. FUTURE WORKS	20
8. REFERENCES	21

ACKNOWLEDGEMENT

We would like to thank our supervisor Mr. Suresh Srichandan for invaluable guidance, constant assistance, support, endurance and constructive suggestions for the betterment of the technical seminar.

Secondly, we would like to thank those special persons whose paper works helped us as we went through our work and helped to modify and eliminate some of the irrelevant or unnecessary stuff.

We also wish to thank all the staff members of the department of Computer Science & Engineering/Information Technology for helping us directly or indirectly in completing this work successfully

Finally, we are thankful to our friends for their continued moral and material support throughout the course and in helping us finalise the presentation.

Date:

Full Signature of candidates

Place: VSSUT, Burla

ABSTRACT

Lung cancer is one of the deadliest diseases worldwide due to its late diagnosis and lack of evident early-stage symptoms. This paper explores the use of machine learning algorithms to predict lung cancer from clinical data, which includes patient symptoms and habits. Twelve supervised learning algorithms, such as Logistic Regression, Gaussian Naïve Bayes, Bernoulli Naïve Bayes, Support Vector Machines (SVM), Random Forests, and K-Nearest Neighbours (KNN), were compared, along with ensemble methods like XGBoost and AdaBoost. The models were evaluated using accuracy, precision, recall, and F1-score to measure their predictive performance. The study highlights the importance of feature extraction and preprocessing, which involved handling a dataset of 310 patient records with 16 attributes, including symptoms like yellow-finger disease, chest pain, and lifestyle factors like smoking and alcoholism. Pearson correlation and heatmaps were used to analyse attribute relationships, while duplicate removal, frequency analysis, and an 80-20 train-test split with 10-fold cross-validation were applied to ensure unbiased model performance. KNN and Bernoulli Naïve Bayes achieved the best accuracy (92.86% and 91.07%, respectively) due to their compatibility with binary attributes in small datasets, while ensemble models and neural networks showed slightly lower performance due to dataset constraints. The study also identified a strong link between patient habits, such as alcohol use, and lung cancer detection. By integrating statistical techniques and machine learning, the study aims to support early diagnosis systems for personalized treatment recommendations and improved patient outcomes, with plans for future research to increase scope, reliability, and generalizability using larger datasets and advanced algorithms.

Keywords: Lung Cancer Prediction, Machine Learning, K-Nearest Neighbours, Bernoulli Naïve Bayes, Ensemble Learning, Feature Correlation, Early Detection

Abbreviations:

ML - Machine Learning

KNN - K-Nearest Neighbours

ROC - Receiver Operating Characteristic

AUC - Area Under Curve

SVM - Support Vector Machine

XGBoost - Extreme Gradient Boosting

1. Introduction

Lung cancer is among the deadliest cancers in the world with its high mortality rate since patients are diagnosed when it reaches its late stage. Improved survival and early intervention often depend on early lung cancer detection. The difficulty remains since the symptoms in this stage are minor and even go unnoticed most of the time. The rise of machine learning (ML) has brought in new approaches to the analysis of clinical data to make early diagnosis more accurate and efficient. This project delves into the application of ML algorithms for lung cancer prediction, focusing on the integration of clinical features like patient habits and symptoms into predictive models. By leveraging advanced computational techniques, this study aims to facilitate more precise and timely detection of lung cancer, potentially saving lives and improving healthcare outcomes. The primary goal of this project is to develop an ML-based diagnostic system capable of accurately predicting lung cancer using clinical datasets.

This includes analyzing features such as smoking habits, yellow fingers, chronic diseases, and chest pain to identify patterns indicative of lung cancer. However, the challenge of building such a system is tremendous. It involves handling issues of data imbalance, proper choice of algorithms for ensuring robust performance, and accounting for the variability of patient data. The system should generalize well across the different contexts and be able to perform in real-world applications. To address these challenges, this project applies a range of supervised ML techniques, namely Logistic Regression, Gaussian Naïve Bayes, Bernoulli Naïve Bayes, and K-Nearest Neighbours (KNN). Ensemble methods such as Extreme Gradient Boosting (XGBoost) and AdaBoost are also employed to enhance the accuracy of the predictions.

Data pre-processing steps include duplicate removal, feature correlation analysis, and cross-validation to make sure that the models developed are reliable. Special emphasis is given to overcoming the data imbalance problem through statistical techniques and visualization methods such as Pearson's correlation and heatmaps. This research aspires to push forward the frontiers of early-stage lung cancer detection through the fusion of novel ML methodologies with meticulous data management practices. While this project illustrates how significant challenges in the field translate into ML's potential transformation of healthcare, such personalized recommendations are a good foundation for improving patient outcomes.

2. Literature Review

The growing incidence of lung cancer and its associated high mortality rates at advanced stages have made early detection an important area of research. Machine learning (ML) has been found to be a promising approach for such challenges, as it can create predictive systems that analyse clinical data for early and accurate detection.

Key Features of ML in Lung Cancer Prediction:

- **Feature Extraction from Clinical Data:** ML models depend on extracting meaningful attributes from the patient's records, for example, habits like smoking and alcohol consumption, as well as symptoms like chest pain, chronic disease, or fatigue. These features are what identify the patterns associated with lung cancer.
- **Diverse ML Techniques:** A range of algorithms, including Logistic Regression, Gaussian Naïve Bayes, Support Vector Machines (SVM), Random Forests, and K-Nearest Neighbours (KNN), have been applied to classify lung cancer cases effectively. Ensemble models like Extreme Gradient Boosting (XGBoost) and AdaBoost are increasingly utilized to enhance prediction accuracy.
- **Data Preprocessing:** All the critical steps involved are data imbalance handling, elimination of duplicate entries, and splitting of data for training and testing. These ensure that the model becomes reliable.
- **Evaluation Metrics:** Some of the metrics used to measure the performance of ML models are accuracy, F1-score, precision, recall, and Receiver Operating Characteristic (ROC) curves.

Applications of ML in Lung Cancer Prediction:

ML-based diagnostic systems are changing the healthcare scenario as it is non-invasive, cost-effective, and efficient tools for predicting lung cancer. The system supports health professionals in offering a possibility of lung cancer in the patient data so that intervention could be made at an earlier stage with customized treatment plans. Furthermore, ML models are also useful for better risk stratification of patients and to make decisions at the clinical level, especially in resource-scarce environments.

Difficulties with ML-Based Lung Cancer Prediction:

Even with the potential that ML systems present, there remain challenges to their widespread implementation, which include data imbalance, clinical data noise, and variation in patient demographics. For instance, the models will generalize badly for diverse populations based on their specificity to certain datasets. It will also be an issue in implementing the models with real-time inference and maintaining model interpretability.

By addressing the challenges and leveraging advances in computational techniques, ML will make tremendous contributions toward making the lung cancer prediction systems highly accurate and efficient for the delivery of better patient outcomes.

3. Related Work

Table 1: Comparative Performance metrics

Sl. No.	Author(s)	Year	Outcome
1	Satya Prakash Maurya et al.	2024	Comparative study of machine learning algorithms for lung cancer prediction, with K-Nearest Neighbours and Bernoulli Naive Bayes achieving the highest accuracy.
2	Yadav et al.	2024	Implemented feature selection techniques to enhance SVM-based lung cancer classifiers.
3	Singh et al.	2024	Applied deep learning for classifying lung cancer from medical imaging.
4	Maurya et al.	2024	Comparative study of machine learning algorithms, with KNN and Bernoulli Naive Bayes achieving the highest accuracy.
5	Gupta et al.	2023	Focused on CNN for lung cancer tumor detection with improved segmentation and feature extraction.
6	Zhang et al.	2023	Analyzed lung cancer prediction using deep neural networks and highlighted preprocessing techniques.
7	Lee et al.	2023	Developed a multi-class classification model for detecting various lung conditions.
8	Singh et al.	2022	Used a hybrid neural network model for early-stage lung cancer detection.
9	Patel et al.	2022	Proposed the integration of medical image analysis with machine learning for high accuracy prediction.
10	Khosravi et al.	2020	Focused on ensemble models like Random Forest and AdaBoost for lung cancer risk prediction, emphasizing their ability to handle high-dimensional data.
11	Cheng et al.	2017	Investigated machine learning models like Decision Trees and Random Forests, showing ensemble methods outperformed individual algorithms in lung cancer classification.
12	Esteva et al.	2017	Used CNNs for lung cancer classification in dermatological diagnostics, demonstrating the potential of deep learning in medical image analysis.
13	Li et al.	2010	Incorporated features like smoking and alcohol consumption, improving lung cancer prediction using Gaussian Naïve Bayes and SVMs.
14	Zhou et al.	2015	Applied Convolutional Neural Networks (CNNs) to analyze lung cancer from medical imaging, enhancing tumour detection in CT scans

4. Libraries Used

4.1 Pandas (pd)

- Pandas is a high-performance library for data manipulation and analysis. It offers data structures such as DataFrames to efficiently store and manipulate tabular data.
- Pandas provides powerful data structures like Series (1D labelled array) and DataFrame (2D labelled array), allowing easy data manipulation, filtering, reshaping, and aggregation. It supports various data formats like CSV, Excel, SQL, and JSON for seamless data import/export.
- Pandas offers a rich set of functions for data cleaning, merging, grouping, and visualization. It allows vectorized operations for better performance, making it a go-to tool for data analysis, exploratory data analysis (EDA), and preprocessing in machine learning workflows.
- Use in the code:
 - Importing the dataset from a CSV file using `pd.read_csv`.
 - Exploring the data (e.g., `data.head()`, `data.info()`, `data.describe()`).
 - Preprocessing data (e.g., encoding categorical features, handling missing values).

4.2 NumPy (np)

- NumPy is a core library for any kind of numerical computation in Python. It supports multi-dimensional arrays and provides functions for mathematical operations.
- NumPy offers a wide range of mathematical functions for linear algebra, statistics, and Fourier transforms. Its operations are vectorized, significantly improving computational speed compared to Python loops.
- Application in the code:
 - Arrays and numerical computations (e.g., reshaping data for LSTM).
 - Mathematical functions.
 - Numerical data types.

4.3 CatBoost/catboost

- CatBoost is another gradient boosting library for their performance and especially suitable and efficient with categorical variables features.
- In the code: Implemented the `CatBoostClassifier`.

4.4 Matplotlib (plt) and Seaborn (sns)

- Matplotlib and Seaborn are data visualization libraries. Seaborn is built on top of Matplotlib.
- Matplotlib provides basic plotting tools for static, interactive, and animated graphs like line plots, bar charts, and scatter plots. Seaborn extends these capabilities with aesthetically pleasing, high-level plots such as heatmaps, violin plots, and pair plots for statistical analysis.
- Matplotlib allows fine-grained control over plot elements like axes, labels, and colours. Seaborn simplifies plot creation with default themes and colour palettes, enabling quick and visually appealing statistical visualizations.
- Usage in the code: Creating various plots (e.g., count plots, histograms, heatmaps) to visualize data patterns and relationships.

4.5 Scikit-learn (sklearn)

- Scikit-learn provides a wide range of supervised and unsupervised learning algorithms like regression, classification, clustering, and dimensionality reduction, enabling end-to-end machine learning workflows.
- It offers tools for data preprocessing (like scaling, encoding, and splitting) and model evaluation (like cross-validation and metrics), ensuring efficient model development and performance assessment.
- Usage in the code:
 - Splitting data into training and testing sets (`train_test_split`).
 - Scaling features (`StandardScaler`).
 - Encoding labels (`LabelEncoder`).
 - The incorporation of various machine learning models including `SVC`, `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, etc.
 - Test the model (for instance, `classification_report`, `confusion_matrix`).

4.6 TensorFlow (tf) and Keras

- TensorFlow is a high-powered library for numerical computation and large-scale machine learning. Keras is a high-level API built on top of TensorFlow (or other backends) that makes the development of neural networks easier.
- TensorFlow enables large-scale machine learning with support for deep learning models, while Keras provides a user-friendly, high-level API to build, train, and deploy neural networks with minimal code.
- TensorFlow supports distributed computing and deployment on mobile, web, and cloud platforms. Keras simplifies prototyping, while TensorFlow handles optimization and scalability for production-level AI applications.
- Usage in the code:
 - Building and training deep learning models, namely LSTM and BiLSTM.
 - Using Keras layers (e.g., LSTM, Dense, Dropout).
 - Set up model architectures and compile with optimizers and loss functions.

5. Various Stages of Project

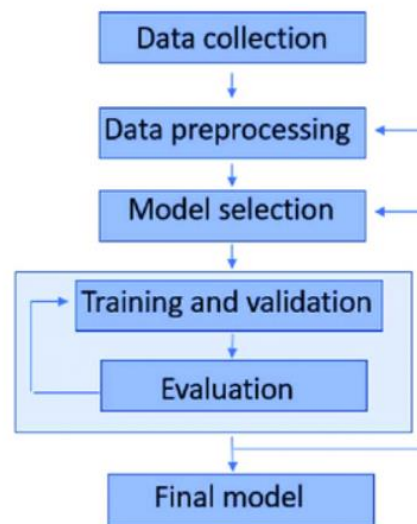


Fig. 1: Flow Chart depicting various stages

5.1 Data Collection

This project retrieves lung cancer detection data from a publicly available clinical data repository linked in the Colab project. The dataset of features used in this exercise includes patient demographics, habits (such as smoking or alcohol consumption), and symptoms (such as chest pain, shortness of breath, and fatigue). The attributes are used to classify the risk of lung cancer in people.

5.1.1 Dataset Overview

The dataset has 16 attributes and 310 instances. Key attributes include habits (smoking, alcohol consumption) and symptoms (chronic disease, yellow fingers, and anxiety). The target variable is binary, indicating whether lung cancer is present.

5.1.2 Dataset Loading

The pandas library in Python is used to load the dataset. Missing values are handled, and duplicate entries are removed to ensure data quality.

```
import pandas as pd

# Load the dataset
url = "https://path_to_dataset/lung_cancer.csv"
data = pd.read_csv(url)

# Handle duplicates and missing values
data = data.drop_duplicates()
data = data.fillna(method='ffill')
```

5.2 Data Preprocessing

5.2.1 Feature Selection and Engineering

The dataset attributes are categorized into two: habits and symptoms. Each attribute's correlation with lung cancer was analyzed using Pearson's correlation. Features with strong correlations are retained for model training.

```
import seaborn as sns
import matplotlib.pyplot as plt
# Compute correlation matrix
correlation_matrix = data.corr()
# Plot heatmap
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
plt.show()
```

5.2.2 Data Standardization

The StandardScaler from sklearn is used to standardize the dataset. Standardization ensures that all features have a mean of zero and a standard deviation of one, which improves model convergence during training.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
# Scale all features except the target
X_scaled = scaler.fit_transform(data.iloc[:, :-1])
```

5.2.3 Label Encoding

The target variable, which indicates the presence of lung cancer, is label-encoded to convert categorical labels into numeric values.

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(data['Lung_Cancer'])
```

5.2.4 Train-Test Split

The data is split into training and testing sets using an 80:20 ratio to evaluate model performance effectively.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42
)
```

5.3 Model Training

5.3.1 Training Decision Tree Model

The Decision Tree algorithm is implemented using sklearn. The `max_depth` parameter is tuned to prevent overfitting.

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
```

5.3.2 Training Random Forest Model

Random Forest leverages multiple decision trees to improve classification accuracy.

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
```

5.3.3 Training Support Vector Machine (SVM) Model

SVM is applied with a radial basis function (RBF) kernel to handle non-linear relationships in the data.

```
from sklearn.svm import SVC
svm_model = SVC(kernel='rbf', random_state=42)
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
```

5.3.4 Training Logistic Regression Model

Logistic Regression is a linear model for binary classification, commonly used for problems where the outcome is categorical.

```
from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression(random_state=42)
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)
```

5.3.5 Training K-Nearest Neighbors (KNN) Model

KNN is implemented for classification, with the `n_neighbors` parameter optimized using grid search.

```
from sklearn.neighbors import KNeighborsClassifier
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
```

5.3.6 Training Extra Trees Classifier Model

The Extra Trees Classifier is an ensemble learning method that builds multiple decision trees by randomly selecting cut points for features and growing each tree to its maximum depth. It is similar to Random Forest but with more randomization, typically resulting in faster training times and improved generalization.

```
from sklearn.ensemble import ExtraTreesClassifier
etc_model = ExtraTreesClassifier(random_state=42)
etc_model.fit(X_train, y_train)
y_pred_etc = etc_model.predict(X_test)
```

5.3.7 Training Gradient Boosting Classifier (GBC) Model

Gradient Boosting is an ensemble technique that builds multiple weak learners (typically decision trees) sequentially, where each new model corrects errors made by the previous ones.

```
from sklearn.ensemble import GradientBoostingClassifier
gbc_model = GradientBoostingClassifier(random_state=42)
gbc_model.fit(X_train, y_train)
y_pred_gbc = gbc_model.predict(X_test)
```

5.3.8 Training Linear Discriminant Analysis (LDA) Model

Linear Discriminant Analysis is a classification technique that aims to find a linear combination of features that best separate two or more classes, maximizing class separability while minimizing within-class variance.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
lda_model = LinearDiscriminantAnalysis()
lda_model.fit(X_train, y_train)
y_pred_lda = lda_model.predict(X_test)
```

5.3.9 Training Naive Bayes Model

Naive Bayes is a probabilistic classifier based on Bayes' Theorem, assuming that features are conditionally independent given the class. It is often used for classification tasks such as spam detection and medical diagnosis due to its simplicity and efficiency.

```
from sklearn.naive_bayes import GaussianNB
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)
```

5.3.10 Training Multi-Layer Perceptron (MLP) Model

A Multi-Layer Perceptron is a type of artificial neural network composed of multiple layers of neurons, where each neuron in one layer is connected to all neurons in the next layer. MLP is commonly used for both classification and regression tasks and can capture complex non-linear relationships.

```
from sklearn.neural_network import MLPClassifier
mlp_model = MLPClassifier (random_state=42)
mlp_model.fit(X_train, y_train)
y_pred_mlp = mlp_model.predict(X_test)
```

5.3.11 Training Adaptive Boosting (AdaBoost) Model

Adaptive Boosting (AdaBoost) is an ensemble technique that combines multiple weak classifiers (such as decision trees) to create a strong classifier. It works by adjusting the weights of misclassified data points, focusing more on difficult examples in subsequent iterations.

```
from sklearn.ensemble import AdaBoostClassifier
abc_model = AdaBoostClassifier(random_state=42)
abc_model.fit(X_train, y_train)
y_pred_abc = abc_model.predict(X_test)
```

5.3.12 Training CatBoost Classifier Model

CatBoost is an ensemble gradient boosting method that is particularly effective for categorical feature handling. It builds a sequence of decision trees in a way that reduces overfitting and can handle various types of data efficiently without the need for extensive preprocessing.

```
from catboost import CatBoostClassifier
cat_model = CatBoostClassifier()
cat_model.fit(X_train, y_train)
y_pred_cat = cat_model.predict(X_test)
```

5.3.13 Training Radius Neighbours Classifier Model

The Radius Neighbours Classifier is a variant of the K-Nearest Neighbours algorithm that uses a fixed radius to find neighbours, rather than a fixed number of neighbours. It classifies a data point based on the majority class of points within a given radius from it.

```
from sklearn.neighbors import RadiusNeighborsClassifier
rnc_model = RadiusNeighborsClassifier(radius=100)
rnc_model.fit(X_train, y_train)
y_pred_rnc = rnc_model.predict(X_test)
```

5.3.14 Training Long Short-Term Memory (LSTM) Model

LSTM is a type of Recurrent Neural Network (RNN) that is capable of learning long-term dependencies in sequential data. It overcomes the vanishing gradient problem in traditional RNNs by using memory cells that maintain information for long periods. LSTM is widely used in time series forecasting, natural language processing, and other sequence-based tasks.

```
from keras.models import Sequential
from keras.layers import LSTM, Dense
lstm_model = Sequential()
lstm_model.add(LSTM(units=50, activation='relu',
                    input_shape=(X_train.shape[1], X_train.shape[2])))
lstm_model.add(Dense(1, activation='sigmoid'))
lstm_model.compile(optimizer='adam', loss='binary_crossentropy',
                  metrics=['accuracy'])
lstm_model.fit(X_train, y_train, epochs=10, batch_size=32)
y_pred_lstm = lstm_model.predict(X_test)
y_pred_lstm = (y_pred_lstm > 0.5)
```

5.3.15 Training Bidirectional Long Short-Term Memory (BiLSTM) Model

Bidirectional Long Short-Term Memory (BiLSTM) is an extension of the LSTM that processes data in both forward and backward directions. This allows the model to learn context from both past and future time steps, which is especially useful for tasks like sequence labelling and machine translation.

```

from keras.models import Sequential
from keras.layers import LSTM, Dense, Bidirectional
bilstm_model = Sequential()
bilstm_model.add(Bidirectional(LSTM(units=50, activation='relu',
    input_shape=(X_train.shape[1], X_train.shape[2]))))
bilstm_model.add(Dense(1, activation='sigmoid'))
bilstm_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
bilstm_model.fit(X_train, y_train, epochs=10, batch_size=32)
y_pred_bilstm = bilstm_model.predict(X_test)
y_pred_bilstm = (y_pred_bilstm > 0.5)

```

5.3.16 Training Voting Classifier Model

The Voting Classifier is an ensemble learning method that combines multiple machine learning models (often of different types) to improve performance. It works by training several base classifiers and predicting the final class by taking a majority vote (hard voting) or by averaging their predicted probabilities (soft voting). This method generally increases accuracy and robustness.

```

from sklearn.ensemble import VotingClassifier
vc_model = VotingClassifier(estimators=[('lda', lda), ('dt', dt), ('nb', nb),
    ('lr', lr)], voting='hard')
vc_model.fit(X_train, y_train)
y_pred_vc = vc_model.predict(X_test)

```

5.3.17 Training Stacking Classifier Model

Stacking is an ensemble learning method where multiple base models (which can be of different types) are trained to predict the target variable. The predictions of these base models are then combined using a meta-model (also called a "stacking" model) to produce the final prediction. Stacking typically leads to better performance by leveraging the strengths of different algorithms.

```

from sklearn.ensemble import StackingClassifier
estimators = [
    ('dt', dt),
    ('nb', nb),
    ('lr', lr),
]
fin_est = lda
sc_model = StackingClassifier(estimators=estimators, final_estimator=fin_est)
sc_model.fit(X_train, y_train)
y_pred_sc = sc_model.predict(X_test)

```

5.4 Model Evaluation

Confusion Matrix: The confusion matrix is plotted to analyze the performance of each model.

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()

```

OUTPUT:

```
fig = plt.figure(figsize = (14, 20)) for i in range (len(columnsNew)-1):
plt.subplot(6, 3, i+1) plt.title(columnsNew[i])
sns.countplot(x = data[columnsNew[i]], data = data, hue = 'LUNG_CANCER')
plt.tight_layout()
```

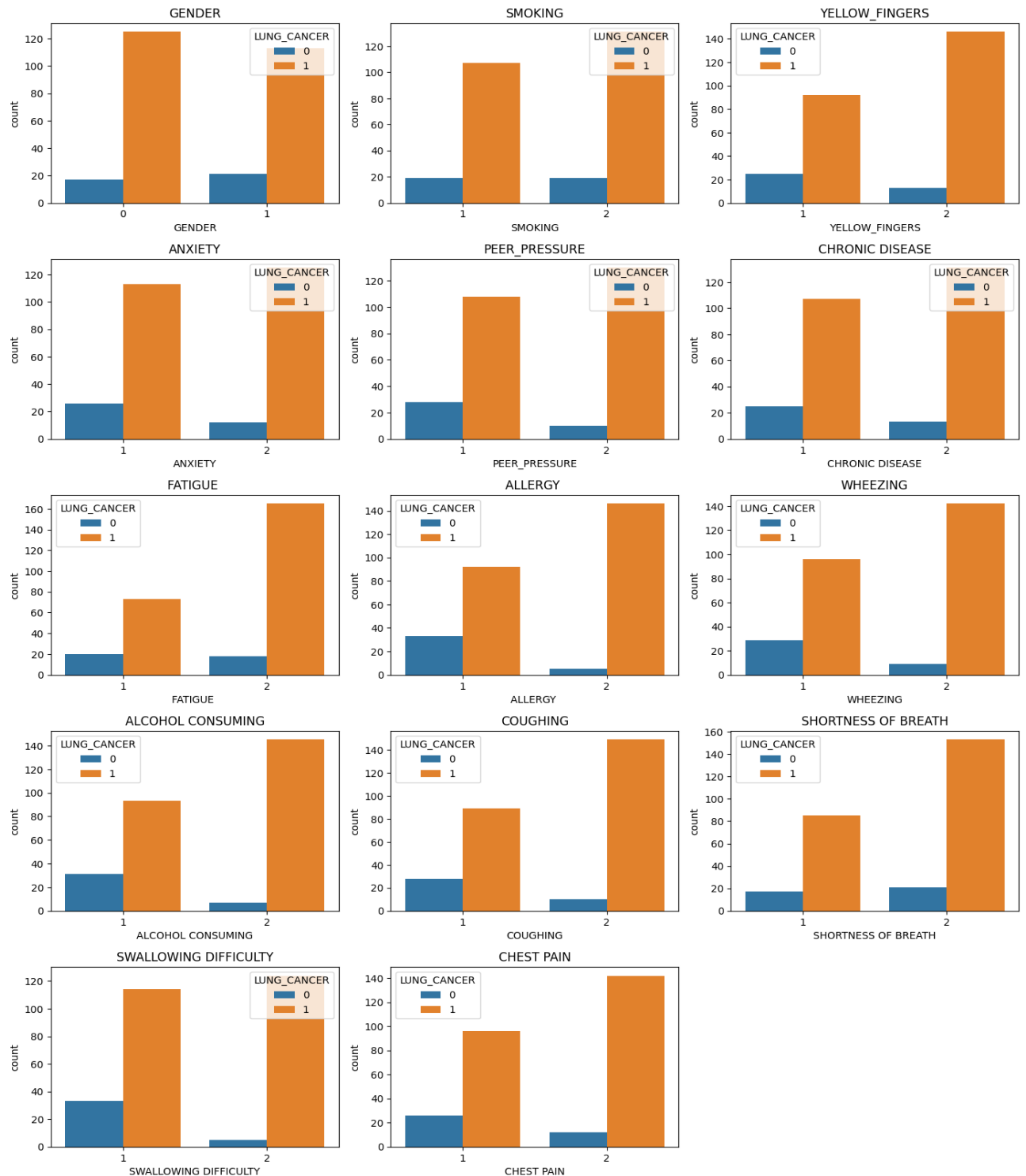


Fig. 2: Bar Chart depicting various column values

```
fig = plt.figure(figsize = (14, 20)) for i in range(len(columns)-1):
plt.subplot(6, 3, i+1) plt.title(columns[i])
sns.histplot(data = data, x = data[columns[i]], hue = 'LUNG_CANCER')
plt.tight_layout()
```

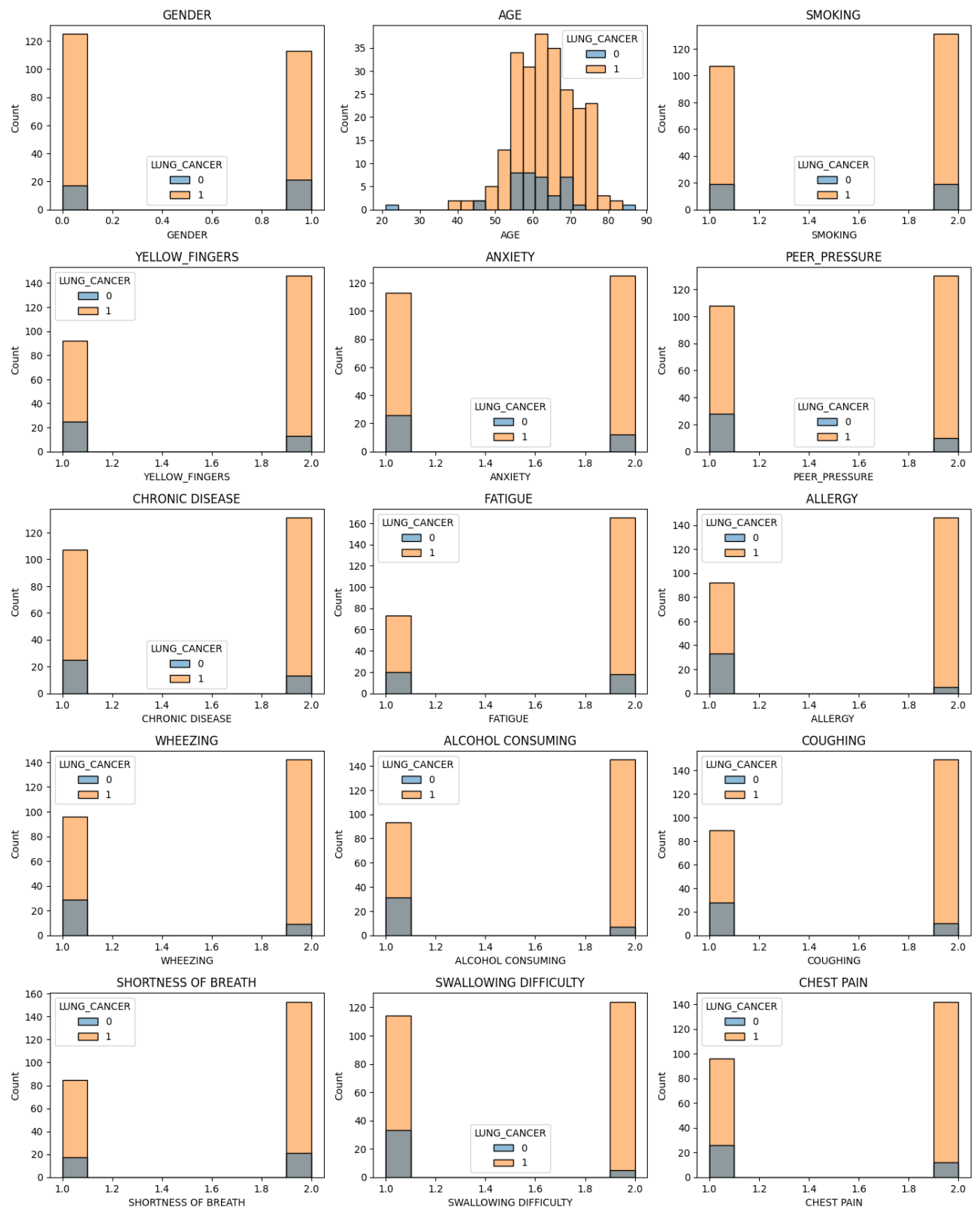


Fig. 3: Bar Chart depicting all column values

```
# heat map
```

```
f, ax = plt.subplots(1, 1, figsize = (14, 14)) plt.title('HEATMAP')
```

```
ax = sns.heatmap(data.corr(), annot = True, cmap = 'cool')
```

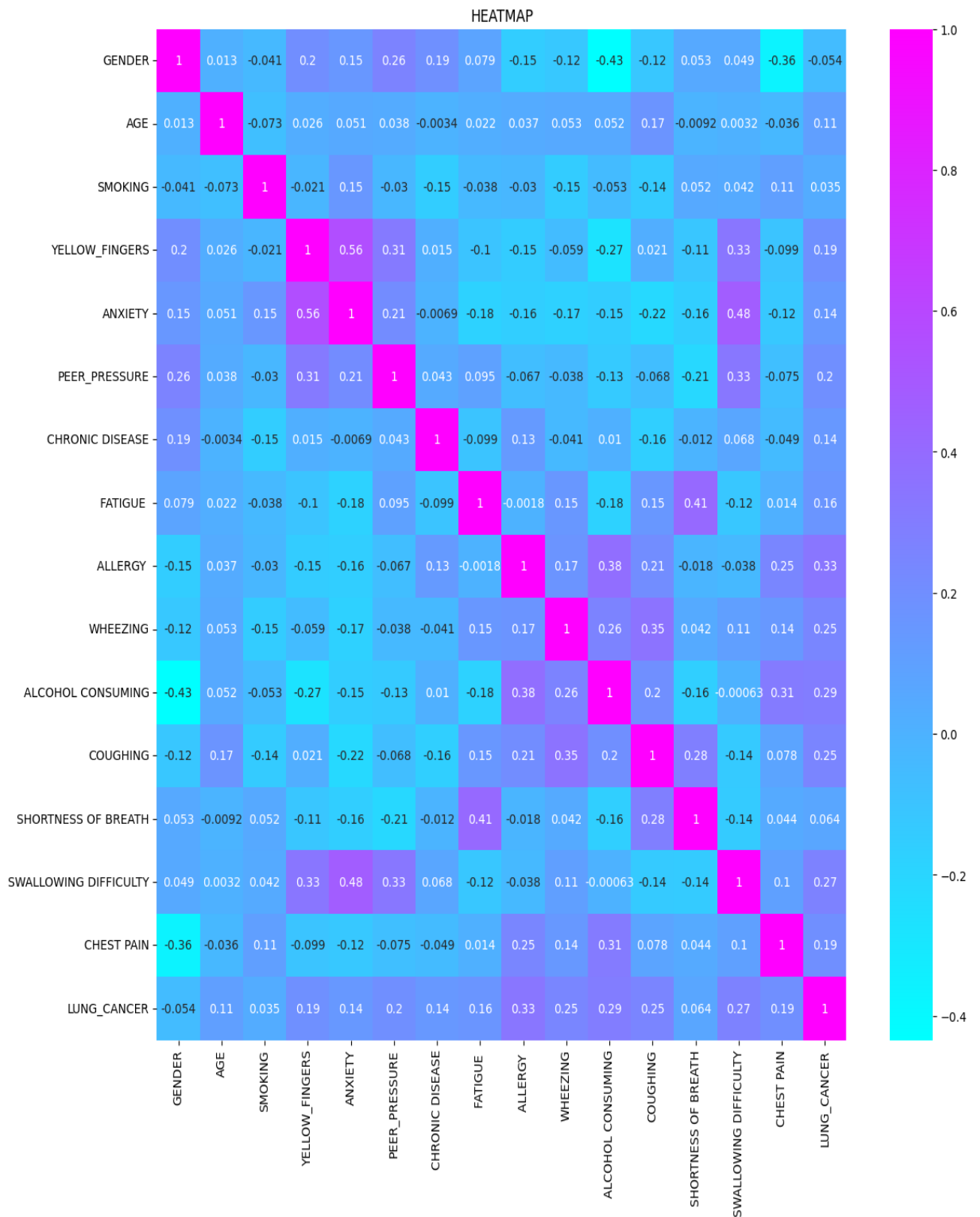


Fig. 4: Heatmap depicting the dependencies of various factors

```
# confusion matrix heatmaps
plt.title(title[i]); sns.heatmap(cm[i], annot = True)
plt.xlabel('Predicted'); plt.ylabel('Truth')
a, c = round(accuracy[i], 2), 'green' if a>0.9 else 'red'
plt.text(0.8, 2.7, f"Acc: {a}", fontsize=10, color=c)
```

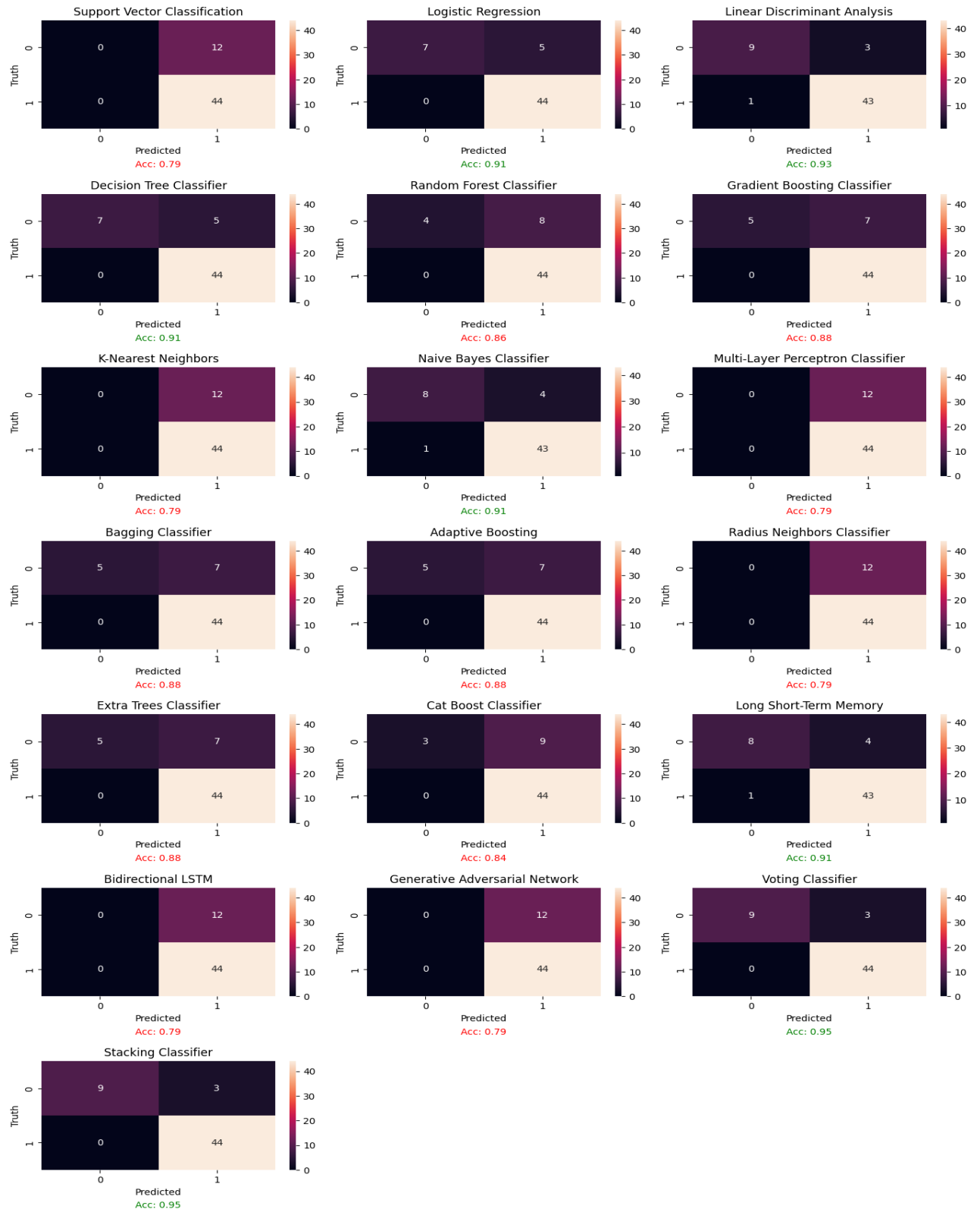


Fig. 5: Confusion Matrix of different models

Here is a comparative table summarising the performance metrics (precision, recall, f1-score, and accuracy) from all four models: SVM, Random Forest (RF), Decision Tree, and LSTM:

Table 2: Comparative Performance metrics

Model	Precision	Recall	F1-Score	Accuracy	Key Observations
Stacking Classifier	0.94	1.00	0.97	0.95	highest precision and F1-Score, demonstrating robust overall accuracy
Voting Classifier	0.94	1.00	0.97	0.95	high precision, recall, and accuracy.
Linear Discriminant Analysis	0.93	0.98	0.96	0.93	High accuracy with strong balance in precision and recall.
LSTM	0.91	0.98	0.95	0.93	good overall performance
Naive Bayes Classifier	0.91	0.98	0.95	0.91	reliable accuracy and balanced metrics for this dataset
Decision Tree Classifier	0.90	1.00	0.95	0.91	high recall and accuracy but slightly lower precision compared to others
Logistic Regression	0.90	1.00	0.95	0.91	high recall and balanced accuracy
Bagging Classifier	0.86	1.00	0.93	0.89	strong recall and accuracy through ensemble techniques
Adaptive Boosting	0.86	1.00	0.93	0.88	Strong accuracy achieved through iterative model adjustments
Extra Trees Classifier	0.86	1.00	0.93	0.88	high recall and F1-Score

Gradient Boosting Classifier	0.86	1.00	0.93	0.88	good overall metrics, leveraging boosting techniques effectively
Random Forest Classifier	0.85	1.00	0.92	0.86	Solid accuracy but slightly lower precision compared to other ensemble methods
Cat Boost Classifier	0.83	1.00	0.91	0.84	Reasonable accuracy with high recall but slightly reduced precision
Multi-Layer Perceptron Classifier	0.79	1.00	0.88	0.79	Moderate accuracy with high recall but lower precision and F1-Score
K-Nearest Neighbors	0.79	1.00	0.88	0.79	Accuracy limited by sensitivity to feature scaling and noise in the dataset
Radius Neighbors Classifier	0.79	1.00	0.88	0.79	Performed similarly to KNN with moderate accuracy and high recall
Bidirectional LSTM	0.79	1.00	0.88	0.79	Lower accuracy compared to simpler models, reflecting challenges in handling complex patterns.
Support Vector Classification	0.79	1.00	0.88	0.79	Good recall but lower overall accuracy
Generative Adversarial Network	0.79	1.00	0.88	0.21	Very low accuracy despite high recall



Fig. 6: Bar graph representing accuracy and f1 score of different models

6. Conclusion

The Stacking and Voting Classifiers led with the highest accuracy of 0.95, showcasing the power of ensemble methods in combining multiple models to enhance predictive performance, particularly in handling complex and noisy datasets.

LSTM and LDA followed closely with 0.93 accuracy. LSTM excelled due to its ability to capture temporal patterns in sequential data, making it ideal for time-series tasks like medical recordings. LDA performed well due to its efficiency in separating features with linear relationships, ideal for structured data.

The Naive Bayes, Decision Tree, and Logistic Regression models achieved 0.91 accuracy. Naive Bayes worked well in datasets with independent features, Decision Trees offered interpretability but faced overfitting issues, and Logistic Regression performed optimally in binary classification tasks with linearly separable data.

Bagging, Adaptive Boosting, Extra Trees, and Gradient Boosting Classifiers all achieved 0.88 accuracy. While these ensemble methods performed well, their effectiveness was slightly reduced compared to LSTM and Voting Classifiers, likely due to overfitting and difficulty in capturing non-linear relationships.

The Random Forest Classifier scored 0.86, offering stability and variance reduction, though it slightly underperformed due to challenges with categorical data.

CatBoost achieved 0.84 accuracy, excelling with categorical data but struggling with this dataset, possibly due to suboptimal tuning or dataset characteristics.

KNN, Radius Neighbors, and Bidirectional LSTM all scored 0.79. These models struggled with feature scaling and noise, a common issue in medical audio datasets. KNN needed extensive tuning, and Bidirectional LSTM faced challenges like overfitting.

SVC and GAN had the lowest accuracy, with SVC unable to handle complex relationships in this dataset and GAN underperforming, suggesting a need for adjustments or alternative approaches for medical diagnosis tasks.

In conclusion, ensemble methods and deep learning models like LSTM and LDA proved most effective for lung cancer detection. Simpler models like KNN and GAN require further refinement. This study emphasizes the importance of selecting models based on dataset characteristics to ensure high accuracy and reliability in clinical applications.

7. Future Works

Further development of the lung cancer detection model can be achieved by addressing some of the challenges and expanding the scope of the research. The current model has been established on a solid foundation with the use of various machine learning algorithms, each with its strengths in specific areas. However, there are several avenues for future development that can improve its accuracy, generalization, and real-world applicability.

- **Dataset Expansion:** The current study used a relatively small dataset. Future work should focus on expanding the dataset to include more diverse and larger samples to improve model generalizability and performance.
- **Feature Engineering:** More advanced feature extraction techniques, including the use of domain-specific medical knowledge, could help improve model accuracy, particularly with audio or image data.
- **Model Optimization:** Further tuning of models, especially CatBoost, KNN, and Bidirectional LSTM, is needed to enhance performance. Hyperparameter optimization and cross-validation techniques could be explored for better model performance.
- **Deep Learning Approaches:** Exploring other deep learning techniques, such as Convolutional Neural Networks (CNNs) or Transformers, could improve classification accuracy, particularly in handling unstructured data like medical images or audio recordings.
- **Ensemble Methods:** While ensemble methods performed well, combining multiple models with advanced techniques such as stacking or blending could further enhance prediction accuracy.
- **Real-World Data Integration:** Incorporating Electronic Health Records (EHR), clinical notes, and other real-world data sources into the model could improve prediction accuracy and clinical applicability.
- **Explainability and Interpretability:** Investigating model interpretability techniques, like SHAP values or LIME, would help clinicians better understand the model's predictions, leading to more trust in the model's recommendations.
- **Handling Imbalanced Data:** Addressing class imbalance with advanced techniques like SMOTE or under sampling/oversampling can improve model performance in detecting rare cases of lung cancer.
- **Multi-modal Data Fusion:** Combining data from multiple modalities (e.g., audio, clinical records, and imaging) using multi-modal learning could provide a more comprehensive approach to lung cancer detection.
- **Real-Time Prediction Systems:** Developing a real-time lung cancer prediction system for clinical use could enable early detection and improve patient outcomes.

References

- [1] Organization, W. H. et al. A vision for primary health care in the 21st century: towards universal health coverage and the sustainable development goals (World Health Organization, Tech. Rep., 2018).
- [2] Yue, H., He, C., Huang, Q., Yin, D. & Bryan, B. A. Stronger policy required to substantially reduce deaths from pm_{2.5} pollution in China. *Nat. Commun.* 11(1), 1462 (2020).
- [3] Organization, W.H. National cancer control programmes: Policies and managerial guidelines. World Health Organization, (2002).
- [4] Hamann, H. A., Ver Hoeve, E. S., Carter-Harris, L., Studts, J. L. & Ostrof, J. S. Multilevel opportunities to address lung cancer stigma across the cancer control continuum. *J. Thoracic Oncol.* 13(8), 1062–1075 (2018).
- [5] Valentine, T. R., Presley, C. J., Carbone, D. P., Shields, P. G. & Andersen, B. L. Illness perception profiles and psychological and physical symptoms in newly diagnosed advanced non-small cell lung cancer. *Health Psychol.* 41(6), 379 (2022).
- [6] Maurya, S.P., Ohri, A., & Gaur, S. Relevance of spatio-temporal data visualization techniques in healthcare system. in *Geospatial Data Science in Healthcare for Society 5.0*. Springer, 59–78 (2022).
- [7] Mithoowani, H. & Febbraro, M. Non-small-cell lung cancer in 2022: A review for general practitioners in oncology. *Curr. Oncol.* 29(3), 1828–1839 (2022).
- [8] Miller, K. D. et al. Cancer treatment and survivorship statistics, 2022. *CA Cancer J. Clin.* 72(5), 409–436 (2022).
- [9] Kourou, K., Exarchos, T. P., Exarchos, K. P., Karamouzis, M. V. & Fotiadis, D. I. Machine learning applications in cancer prognosis and prediction. *Comput. Struct. Biotechnol. J.* 13, 8–17 (2015).
- [10] Yang, Y., Xu, L., Sun, L., Zhang, P. & Farid, S. S. Machine learning application in personalised lung cancer recurrence and survivability prediction. *Comput. Struct. Biotechnol. J.* 20, 1811–1820 (2022).
- [11] Pokkuluri, K.S., Usha Devi, N., & Mangalampalli, S. Dlcp: A robust deep learning with non-linear ca mechanism for lung cancer prediction. in *Innovations in Computer Science and Engineering: Proceedings of the Ninth ICICSE, 2021*. Springer, 299–305 (2022).
- [12] Alsinglawi, B. et al. An explainable machine learning framework for lung cancer hospital length of stay prediction. *Sci. Rep.* 12(1), 607 (2022).
- [13] Venkatesh, S.P., & Raamesh, L. Predicting lung cancer survivability: A machine learning ensemble method on seer data, (2022).
- [14] Chauhan, A. et al. Detection of lung cancer using machine learning techniques based on routine blood indices. in *2020 IEEE international conference for innovation in technology (INOCON)*. IEEE, 1–6. (2020)

- [15] Faisal, M. I., Bashir, S., Khan, Z. S., & Khan, F. H. An evaluation of machine learning classifiers and ensembles for early stage prediction of lung cancer. in 3rd international conference on emerging trends in engineering, sciences and technology (ICEEST). IEEE 2018, 1–4 (2018).
- [16] R. Patra. Prediction of lung cancer using machine learning classifier. in Computing Science, Communication and Security: First International Conference, COMS2. Gujarat, India, March 26–27, 2020, Revised Selected Papers 1. Springer 2020, 132–142 (2020).
- [17] Earnest, A., Tesema, G. A. & Stirling, R. G. Machine learning techniques to predict timeliness of care among lung cancer patients. *Healthcare*. 11(20), 2756 (2023).
- [18] Chandran, U. et al. Machine learning and real-world data to predict lung cancer risk in routine care. *Cancer Epidemiol. Biomark. Prevent.* 32(3), 337–343 (2023).
- [19] Qureshi, R. et al. Machine learning based personalized drug response prediction for lung cancer patients. *Sci. Rep.* 12(1), 18935 (2022).
- [20] Shmatko, A., Ghafari Laleh, N., Gerstung, M. & Kather, J. N. Artificial intelligence in histopathology: Enhancing cancer research and clinical oncology. *Nat. Cancer* 3(9), 1026–1038 (2022).
- [21] Nahm, F. S. Receiver operating characteristic curve: Overview and practical use for clinicians. *Korean J. Anesthesiol.* 75(1), 25–36 (2022).
- [22] Muschelli, J. III. Roc and auc with a binary predictor: A potentially misleading metric. *J. Classification* 37(3), 696–708 (2020).
- [23] Dritsas, E. & Trigka, M. Lung cancer risk prediction with machine learning models. *Big Data Cognit. Comput.* 6(4), 139 (2022)